



TRƯỜNG ĐẠI HỌC THƯƠNG MẠI

Khoa HTTT Kinh tế và TMĐT

Bộ môn Công nghệ thông tin

Chương 3

THIẾT KẾ CA KIỂM THỬ



Nội dung

1. Kiểm thử hộp đen (kiểm thử chức năng)
 - Kiểm thử lớp tương đương
 - Kiểm thử giá trị biên
 - Kiểm thử bảng quyết định
 - Kiểm thử dựa trên đặc tả use cases
2. Kiểm thử hộp trắng (kiểm thử cấu trúc)
 - Kiểm thử luồng điều khiển
 - Kiểm thử luồng dữ liệu



PHẦN I: KIỂM THỬ HỘP ĐEN



Tổng quan kiểm thử hộp đen

- Kiểm thử hộp đen phù hợp cho mọi mức kiểm thử:
 - Kiểm thử mức đơn vị
 - Kiểm thử mức tích hợp
 - Kiểm thử mức hệ thống
 - Kiểm thử chấp nhận
- Đầu vào: đặc tả yêu cầu của phần mềm
- Người kiểm thử không cần có kiến thức về thuật giải, cấu trúc dữ liệu, mã nguồn



Quy trình kiểm thử hộp đen

1. Phân tích đặc tả yêu cầu chức năng mà phần mềm cần thực hiện
2. Áp dụng một kỹ thuật thiết kế test cases cho phần mềm đó
3. Mô tả test cases với 3 thông tin:
 - Giá trị dữ liệu nhập vào để phần mềm xử lý
 - Trạng thái của phần mềm cần để thực hiện test cases
 - Giá trị dữ liệu xuất mà phần mềm cần tạo được
4. Thực hiện kiểm thử với các test cases trên
5. So sánh kết quả



Một số kỹ thuật kiểm thử hộp đen

- Kỹ thuật phân lớp tương đương
- Kỹ thuật phân tích các giá trị biên
- Kỹ thuật dùng các bảng quyết định
- Kỹ thuật dựa trên đặc tả use cases



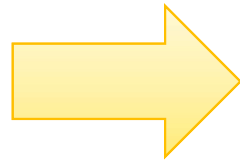
Kiểm thử lớp tương đương

- Equivalent Class Partitioning (ECP)
- Equivalent Class Testing (ECT)
- Phân các test cases thành các lớp (nhóm) khác nhau sao cho:
 - Các test cases trong mỗi lớp sẽ kích hoạt phần mềm thực hiện cùng một hành vi
- Mỗi nhóm test cases thỏa mãn tiêu chuẩn trên được gọi là một lớp tương đương
- Chỉ cần chọn một test cases trong mỗi nhóm làm đại diện cho nhóm này để thực hiện kiểm thử phần mềm
 - Nếu một test case trong lớp tương đương nào đó gây lỗi thì các test cases khác trong lớp này cũng gây lỗi như vậy
 - Nếu một test case trong lớp tương đương nào đó không gây lỗi thì các test cases khác trong lớp này cũng sẽ không gây lỗi.



Kiểm thử lớp tương đương

- Lý do:
 - Cảm giác kiểm thử hết
 - Tránh dư thừa
- Các lớp tương đương tạo thành một phân hoạch của miền dữ liệu
 - Hợp của tất cả các lớp bằng miền đầu vào
 - Cảm giác đã kiểm thử hết
 - Hai lớp bất kỳ không giao nhau
 - Không dư thừa



1. Giảm rất nhiều test cases cần định nghĩa và kiểm thử
2. Chất lượng kiểm thử không bị giảm sút đáng kể so với vét cạn



Các lớp tương đương

- Ý tưởng của ECT là chỉ kiểm thử với một phần tử của mỗi miền tương đương
 - Giảm rất nhiều dư thừa tiềm tàng nếu các lớp tương đương được chọn hợp lý
- Mấu chốt là làm sao chọn được quan hệ tương đương để từ đó xác định được các lớp tương đương (phân hoạch)



Chọn phân hoạch

- Thường là “thủ công” (craft):
 - Không dựa trên mã nguồn, chỉ dựa trên đặc tả
 - Cần hiểu biết về miền xác định, thường không thể xác định dựa vào đặc tả thiết kế giao diện
 - Phải hiểu đầu vào phụ thuộc nhau như thế nào



Ví dụ

- Xét chương trình P có ba biến đầu vào: a, b và c với các miền xác định là A, B, and C.
- Phân hoạch của các miền này giả sử là:

$$A = A1 \cup A2 \cup A3$$

$$B = B1 \cup B2 \cup B3 \cup B4$$

$$C = C1 \cup C2$$



Ví dụ (tiếp)

- Gọi ai thuộc a_i là một phần tử đại diện của lớp
 - Ví dụ lấy phần tử giữa của 1 khoảng
- Tương tự có b_i và c_i .
- Các ca kiểm thử sẽ được xây dựng từ các phần tử đại diện này
- Ý tưởng ở đây là phần tử đại diện này cũng tốt như các phần tử khác ở trong cùng lớp tương đương đó

ECT yếu

- ECT yếu chỉ lấy tất cả các phần tử đại diện ít nhất một lần
- Số ca kiểm thử tối thiểu sẽ bằng số lớp của phân hoạch có nhiều tập con nhất
 - Trong ví dụ trước là 4

#	a	b	c
WE1	a_1	b_1	c_1
WE2	a_2	b_2	c_2
WE3	a_3	b_3	c_1
WE4	a_1	b_4	c_2



ECT mạnh

- ECT mạnh dựa trên tích Đề-các của các lớp con
- Với ví dụ trước ta có:
$$3 * 4 * 2 = 24 \text{ ca kiểm thử}$$
- Cách này xét đến tất cả các tương tác của các giá trị đại diện



#	a	b	c
SE1	a1	b1	c1
SE2	a1	b1	c2
SE3	a1	b2	c1
SE4	a1	b2	c1
SE5	a1	b3	c1
SE6	a1	b3	c2
SE7	a1	b4	c1
SE8	a1	b4	c2
SE9	a2	b1	c1
SE10	a2	b1	c2
SE11	a2	b2	c1
SE12	a2	b2	c2
SE13	a2	b3	c1
SE14	a2	b3	c2
SE15	a2	b4	c1
SE16	a2	b4	c2
SE17	a3	b1	c1
SE18	a3	b1	c2
SE19	a3	b2	c1
SE20	a3	b2	c2
SE21	a3	b3	c1
SE22	a3	b3	c2
SE23	a3	b4	c1
SE24	a3	b4	c2



ECT truyền thống

- Chỉ phân biệt lớp giá trị hợp lệ và không hợp lệ



Mở rộng cho miền đầu ra

- Hai phương pháp ECT yếu và mạnh trên đều có thể mở rộng cho các biến đầu ra



Các ca kiểm thử ECT cho đầu ra bài toán Triangle

- Miền đầu ra có các giá trị:
 - Không là tam giác
 - Tam giác đều
 - Tam giác cân
 - Tam giác thường
- Chúng ta sử dụng chúng để xác định lớp tương đương
 - $R1 = \{ \langle a, b, c \rangle \mid \text{ba cạnh } a, b, c \text{ tạo thành tam giác cân} \}$
 - $R2 = \{ \langle a, b, c \rangle \mid \text{ba cạnh } a, b, c \text{ tạo thành tam giác đều} \}$
 - $R3 = \{ \langle a, b, c \rangle \mid \text{ba cạnh } a, b, c \text{ tạo thành tam giác thường} \}$
 - $R4 = \{ \langle a, b, c \rangle \mid \text{ba cạnh } a, b, c \text{ không tạo thành tam giác} \}$
- Các lớp này cho các ca kiểm thử sau:



Một bộ ca kiểm thử dựa trên đầu ra

TT	a	b	c	Kết quả mong đợi
OE1	5	5	5	Cân
OE2	2	2	3	Đều
OE3	3	4	5	Thường
OE4	4	1	2	Không là tam giác



Một bộ ca kiểm thử dựa trên đầu vào

- Dựa trên đầu vào sẽ cho tập ca kiểm thử lớn hơn.

- Các lớp tương đương:

$$D1 = \{ \langle a, b, c \rangle \mid a = b = c \}$$

$$D2 = \{ \langle a, b, c \rangle \mid a = b, a \neq c \}$$

$$D3 = \{ \langle a, b, c \rangle \mid a = c, a \neq b \}$$

$$D4 = \{ \langle a, b, c \rangle \mid b = c, a \neq b \}$$

$$D5 = \{ \langle a, b, c \rangle \mid a \neq b, a \neq c, b \neq c \}$$

- Có thể thêm các lớp để loại trừ ba cạnh không tạo thành tam giác

$$D6 = \{ \langle a, b, c \rangle \mid a \geq b + c \}$$

$$D7 = \{ \langle a, b, c \rangle \mid b \geq a + c \}$$

$$D8 = \{ \langle a, b, c \rangle \mid c \geq a + b \}$$

- D6 có thể tách tiếp thành

$$D6' = \{ \langle a, b, c \rangle \mid a = b + c \} \text{ và } D6'' = \{ \langle a, b, c \rangle \mid a > b + c \}$$



Một bộ ca kiểm thử dựa trên đầu vào cho NextDate

- NextDate là hàm của ba biên với miền xác định:

$$1 \leq \text{month} \leq 12$$

$$1 \leq \text{day} \leq 31$$

$$1812 \leq \text{year} \leq 2012$$



Các ca ECT truyền thống

- Các lớp dữ liệu hợp lệ:

$M1 = \{\text{month} \mid 1 \leq \text{month} \leq 12\}$

$D1 = \{\text{day} \mid 1 \leq \text{day} \leq 31\}$

$Y1 = \{\text{year} \mid 1812 \leq \text{year} \leq 2012\}$

- Các lớp dữ liệu không hợp lệ:

$M2 = \{\text{month} \mid \text{month} < 1\}$

$M3 = \{\text{month} \mid \text{month} > 12\}$

$D2 = \{\text{day} \mid \text{day} < 1\}$

$D3 = \{\text{day} \mid \text{day} > 31\}$

$Y2 = \{\text{year} \mid \text{year} < 1812\}$

$Y3 = \{\text{year} \mid \text{year} > 2012\}$



Các ca ECT truyền thống

	Month	Day	Year	KQ mong đợi
TE1	6	15	1912	6/16/1912
TE2	-1	15	1912	Invalid
TE3	13	15	1912	Invalid
TE4	6	-1	1912	Invalid
TE5	6	32	1912	Invalid
TE6	6	15	1811	Invalid
TE7	6	15	2013	Invalid



Một phân hoạch mịn hơn

- Nếu chọn quan hệ tương đương khéo hơn ta có các lớp tương đương có ích hơn

$M1 = \{\text{month} \mid \text{tháng có 30 ngày}\}$

$M2 = \{\text{month} \mid \text{tháng có 31 ngày}\}$

$M3 = \{\text{month} \mid \text{tháng Hai}\}$

$D1 = \{\text{day} \mid 1 \leq \text{day} \leq 28\}$

$D2 = \{\text{day} \mid \text{day} = 29\}$

$D3 = \{\text{day} \mid \text{day} = 30\}$

$D4 = \{\text{day} \mid \text{day} = 31\}$

$Y1 = \{\text{year} \mid \text{year} = 1900\}$

$Y2 = \{\text{year} \mid 1812 \leq \text{year} \leq 2012 \text{ AND } \text{year} \neq 1900 \text{ AND } (0 = \text{year mod } 4)\}$

$Y3 = \{\text{year} \mid 1812 \leq \text{year} \leq 2012 \text{ AND } (0 \neq \text{year mod } 4)\}$



Các ca ECT mạnh

CASE ID	Month	Day	Year	Output
SE1	6	14	1900	6/15/1900
SE2	6	14	1912	6/15/1912
SE3	6	14	1913	6/15/1913
SE4	6	29	1900	6/30/1900
SE5	6	29	1912	6/30/1912
SE6	6	29	1913	6/30/1913
SE7	6	30	1900	7/1/1900
SE8	6	30	1912	7/1/1912
SE9	6	30	1913	7/1/1913
SE10	6	31	1900	ERROR
SE11	6	31	1912	ERROR
SE12	6	31	1913	ERROR
SE13	7	14	1900	7/15/1900
SE14	7	14	1912	7/15/1912
SE15	7	14	1913	7/15/1913
SE16	7	29	1900	7/30/1900
SE17	7	29	1912	7/30/1912
SE18	7	29	1913	7/30/1913



Các ca ECT mạnh (tiếp)

CASE ID	Month	Day	Year	Output
SE19	7	30	1900	7/31/1900
SE20	7	30	1912	7/31/1912
SE21	7	30	1913	7/31/1913
SE22	7	31	1900	8/1/1900
SE23	7	31	1912	8/1/1912
SE24	7	31	1913	8/1/1913
SE25	2	14	1900	2/15/1900
SE26	2	14	1912	2/15/1912
SE27	2	14	1913	2/15/1913
SE28	2	29	1900	ERROR
SE29	2	29	1912	3/1/1912
SE30	2	29	1913	ERROR
SE31	2	30	1900	ERROR
SE132	2	30	1912	ERROR
SE33	2	30	1913	ERROR
SE34	2	31	1900	ERROR
SE35	2	31	1912	ERROR
SE36	2	31	1913	ERROR



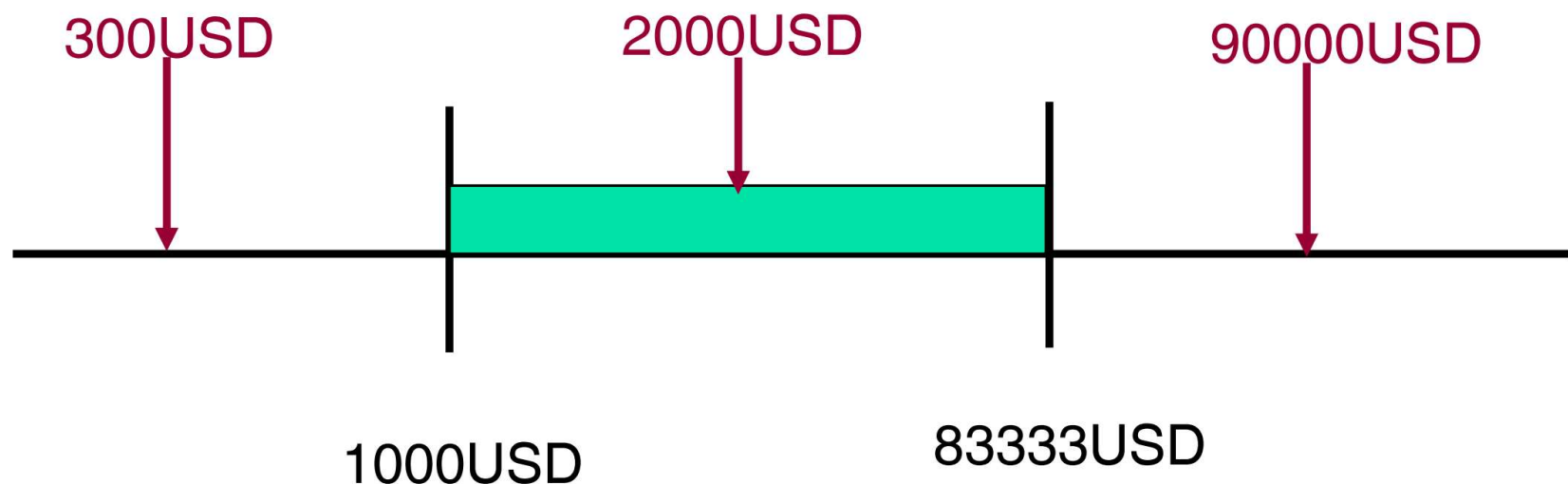
Hướng dẫn và cân nhắc

- Các loại lớp tương đương: ECT cho đầu vào vs ECT cho đầu ra
 - ECT truyền thống
 - ECT yếu
 - ECT mạnh
- ECT truyền thống khá yếu, kém hơn ECT yếu, bản thân đã kém hơn ECT mạnh
- Nếu cần kiểm tra dữ liệu không hợp lệ thì cần thêm các lớp tương đương ngoài khoảng xác định
- ECT phù hợp với dữ liệu đầu vào là khoảng hoặc tập các giá trị rời rạc



Hướng dẫn và cân nhắc (tiếp)

- Nếu lớp tương đương được xác định bởi tập các dữ liệu nhập là số thực/số nguyên liên tục, ta chọn 1 test case đại diện cho giá trị nhập hợp lệ và nếu muốn thì thêm 2 test cases miêu tả giá trị nhập không hợp lệ nằm phía dưới và phía trên khoảng trị hợp lệ





Hướng dẫn và cân nhắc (tiếp)

- Nếu lớp tương đương được xác định bởi tập các dữ liệu dạng liệt kê rời rạc và không có mối quan hệ lẫn nhau gồm 1 trị hợp lệ và nhiều trị không hợp lệ, ta chọn 1 test case đại diện cho giá trị nhập hợp lệ và nếu muốn thì thêm 2 test cases miêu tả giá trị nhập không hợp lệ nào đó.
- Nếu lớp tương đương được xác định bởi tập các dữ liệu dạng liệt kê rời rạc và không có mối quan hệ lẫn nhau gồm N trị hợp lệ và M trị không hợp lệ, ta chọn 1 test case đại diện cho giá trị nhập hợp lệ và nếu muốn thì thêm 2 test cases miêu tả giá trị nhập không hợp lệ nào đó.

Số lượng các test cases cho mỗi lớp tương đương là từ 1 đến 3



Hướng dẫn và cân nhắc (tiếp)

- Trong trường hợp chương trình có nhiều loại dữ liệu nhập (nhiều biến, nhiều miền xác định), số lượng test cases nhiều. Để giảm số lượng test cases mà vẫn đảm bảo chất lượng kiểm thử, cách chọn các test cases:
 - 1 test case cho tổ hợp các giá trị hợp lệ
 - N test cases cho tổ hợp các giá trị trong đó có 1 giá trị không hợp lệ, các giá trị còn lại hợp lệ, nên thay đổi các giá trị hợp lệ trong tổ hợp cho mỗi test case



Bài tập

- Ứng dụng kiểm thử lớp tương đương cho một số bài toán ví dụ:
 - Tính lương nhân viên theo mức lương tối thiểu vùng
- Để phân lớp cần:*
 - + *Tìm hiểu mức lương tối thiểu vùng để phân lớp*
 - + *Giờ làm tối đa, tối thiểu trong ngày, tháng*
- Xác định lớp tương đương và xây dựng test case cho ETC truyền thống, yếu, mạnh***
- Bài toán “quản lý nguồn nhân lực”
- Bài toán “xét đơn xin cầm cố nhà”



-
- + Tính lương theo lương tối thiểu vùng của người lao động
 - + Tuyển dụng
 - + Xác định khách hàng tiềm năng theo khu vực, thu nhập, giới tính, độ tuổi
 - => Phân lớp tương đương
 - => Xây dựng các kiểm thử



Nội dung

1. Kiểm thử hộp đen (kiểm thử chức năng)
 - Kiểm thử lớp tương đương
 - **Kiểm thử giá trị biên**
 - Kiểm thử bảng quyết định
 - Kiểm thử dựa trên đặc tả use cases
2. Kiểm thử hộp trắng (kiểm thử cấu trúc)
 - Kiểm thử luồng điều khiển
 - Kiểm thử luồng dữ liệu



Kỹ thuật kiểm thử phân tích giá trị biên

- Các chương trình có thể coi là một hàm (toán học)
 - Các đầu vào chương trình là **miền xác định** của hàm
 - Các đầu ra là **miền giá trị** của hàm
- Phân tích giá trị biên (boundary value analysis - BVA) là kỹ thuật kiểm thử hàm phổ biến nhất
- Mục tiêu của kiểm thử hàm là sử dụng kiến thức về hàm để xác định các ca kiểm thử
 - Trước kia chủ yếu tập trung vào **miền xác định**, nhưng nay đã dựa trên cả **miền giá trị** của hàm để xác định ca kiểm thử



Kỹ thuật kiểm thử phân tích giá trị biên

- Chỉ phù hợp với các lớp tương đương được xác định bởi các giá trị liên tục (số nguyên, số thực)
- Không phù hợp với các lớp tương đương được xác định bởi các giá trị liệt kê mà không có mối quan hệ lẫn nhau.



Phân tích giá trị biên (BVA)

- Boundary Value Analysis (BVA)
- Phân tích giá trị biên tập trung vào biên của miền xác định để xây dựng ca kiểm thử
- Lý do là lỗi thường xảy ra ở gần các giá trị biên này
- Chương trình viết bằng ngôn ngữ không có kiểm tra kiểu mạnh càng cần kiểm thử giá trị biên
 - Javascript, php, Visual Basic



Đầu vào hợp lệ của chương trình P

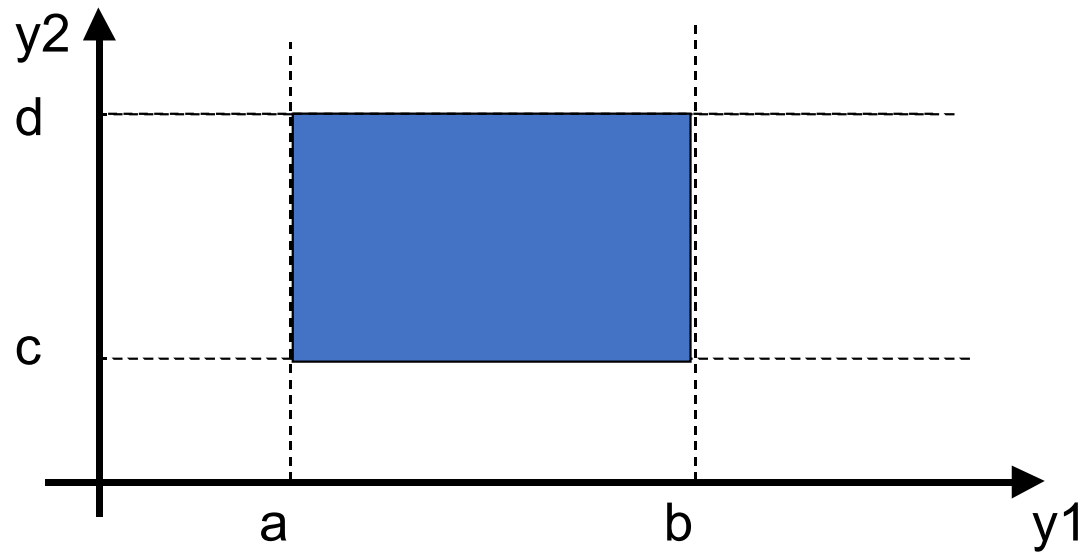
- Từ đây chúng ta giả sử có chương trình P nhận hai biến đầu vào là y_1 và y_2 thỏa mãn $a \leq y_1 \leq b$ and $c \leq y_2 \leq d$

$P(y_1, y_2)$ where $a \leq y_1 \leq b, c \leq y_2 \leq d$

- Chương trình nhận n đầu vào sẽ có không gian đầu vào n chiều



Miền xác định





Chọn giá trị (1)

- Phân tích giá trị biên (thường) sẽ chọn các giá trị:
 - Giá trị nhỏ nhất
 - Ngay trên giá trị nhỏ nhất
 - Một giá trị bình thường
 - Ngay dưới giá trị lớn nhất
 - Giá trị lớn nhất
- Ví dụ:
 - $a \leq y_1 \leq b$ thì sẽ chọn $a, a+1, a+b/2, b-1, b$.



Chọn giá trị (2)

- Phân tích giá trị biên (mạnh) sẽ chọn các giá trị:
 - Giá trị nhỏ nhất
 - Ngay dưới giá trị nhỏ nhất
 - Ngay trên giá trị nhỏ nhất
 - Một giá trị bình thường
 - Ngay dưới giá trị lớn nhất
 - Giá trị lớn nhất
 - Ngay trên giá trị lớn nhất
- Ví dụ:
 - $a \leq y_1 \leq b$ thì sẽ chọn $a-1, a, a+1, a+b/2, b-1, b, b+1$.

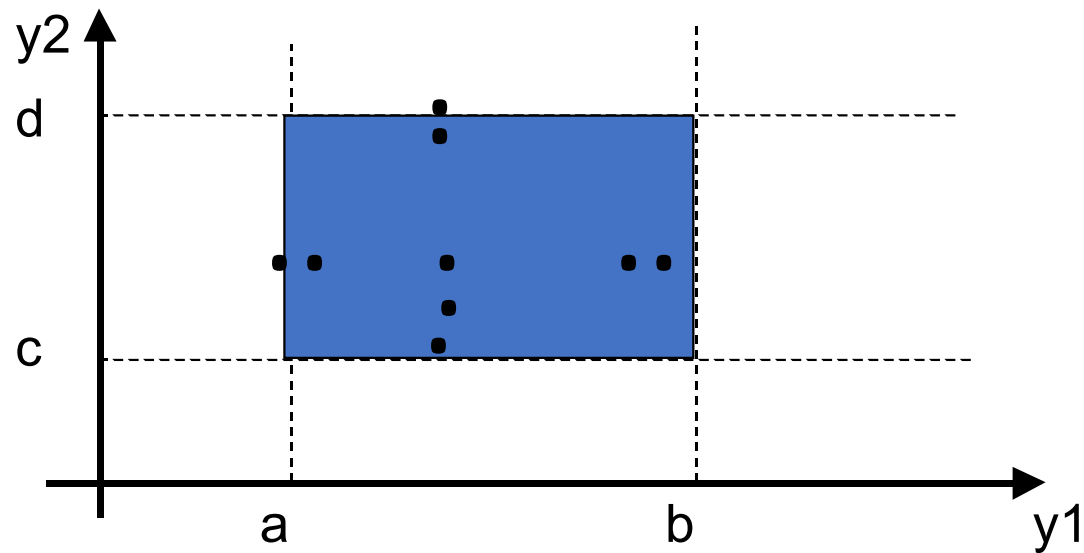


Quy trình thiết kế test cases dựa trên VBA

1. Nhận ra các lớp tương đương dựa trên phân tích đặc tả yêu cầu phần mềm
2. Xác định 2 biên của mỗi lớp tương đương
3. Chọn các test cases cho mỗi biên của mỗi lớp tương đương
 - 1 test case cho giá trị biên (một số test cases cho tổ hợp các giá trị biên, nếu >1 biến)
 - 1 test case ngay dưới biên, 1 test case ngay trên biên (một số test cases cho tổ hợp các giá trị ngay dưới và ngay trên biên, nếu >1 biến)
 - 1 test case cho giá trị thường (tổ hợp ...)



Các ca kiểm thử theo phân tích giá trị biên (thường)



$$T = \{ \langle y_{1nom}, y_{2min} \rangle, \langle y_{1nom}, y_{2min+} \rangle, \langle y_{1nom}, y_{2nom} \rangle, \langle y_{1nom}, y_{2max-} \rangle, \\ \langle y_{1nom}, y_{2max+} \rangle, \langle y_{1min}, y_{2nom} \rangle, \langle y_{1min+}, y_{2nom} \rangle, \langle y_{1max-}, y_{2nom} \rangle, \\ \langle y_{1max}, y_{2nom} \rangle \}$$

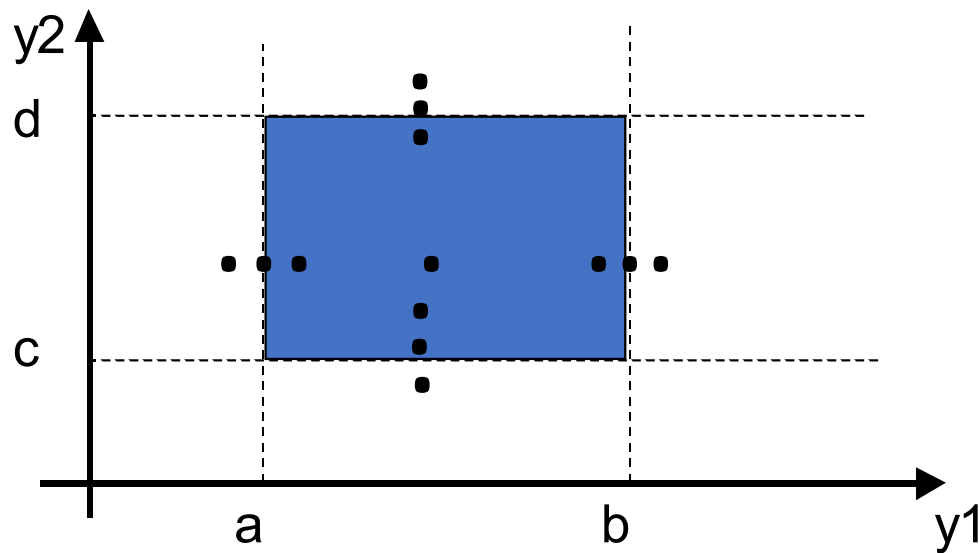


Các ca kiểm thử cho Triangle

Case #	A	b	c	Expected Output
1	100	100	1	Isosceles/Cân
2	100	100	2	Isosceles
3	100	100	100	Equilateral/Đều
4	100	100	199	Isosceles
5	100	100	200	Not a Triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	100	100	Equilateral
9	100	199	100	Isosceles
10	100	200	100	Not a Triangle
11	1	100	100	Isosceles
12	2	100	100	Isosceles
13	100	100	100	Equilateral
14	199	100	100	Isosceles
15	200	100	100	Not a Triangle



Các ca kiểm thử biên mạnh cho P





Kiểm thử giá trị đặc biệt

- Kiểm thử giá trị đặc biệt là phương pháp được thực hiện nhiều nhất trên thực tế, nó cũng trực quan nhất, và không có dạng cố định nhất
- Sử dụng kỹ nghệ và kiến thức miền ứng dụng để phán đoán và đưa ra ca kiểm thử
- Mặc dù mang tính chủ quan cao, đây vẫn là phương pháp hiệu quả để phát hiện khiếm khuyết của chương trình



Hướng dẫn áp dụng BVA

- Có thể áp dụng BVA cho miền giá trị (đầu ra), hay các biến khác trong chương trình như biến đếm vòng lặp, biến trung gian, chỉ số mảng, con trỏ,...
- Chỉ phù hợp với các lớp tương đương được xác định bởi **các giá trị liên tục (số nguyên, số thực)**
- Không phù hợp với các lớp tương đương được xác định bởi **các giá trị liệt kê mà không có mối quan hệ lẫn nhau**.
- Ví dụ, bài toán “xét đơn xin cầm cố nhà” chỉ áp dụng cho 2 biến có giá trị liên tục là “thu nhập” và “số lượng nhà xin cầm cố”, (biến “đối tượng đang đơn” và “loại nhà xin cầm cố” là biến nhận các giá trị liệt kê)



Bài toán chia hoa hồng

- Một người bán hàng chuyên bán khóa, bóng và nòng súng trường cho một cửa hàng.
- Giá của Khóa = 45, bóng = 30, và nòng = 25 đô
- Mỗi người bán hàng phải bán ít nhất một bộ đầy đủ mỗi tháng (100 đô)
- Người bán giỏi nhất bán được 70 khóa, 80 bóng, 90 nòng một tháng

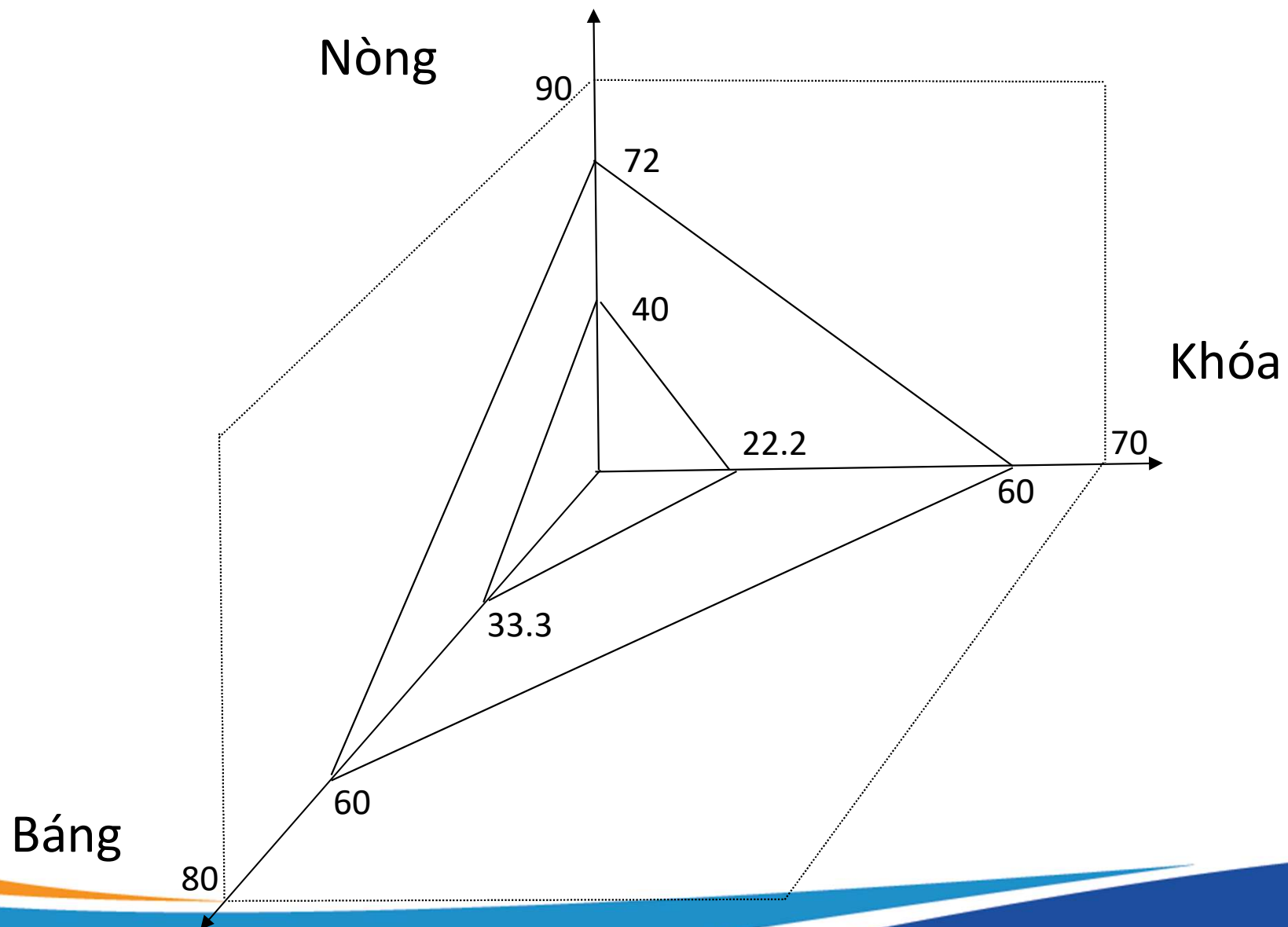


Bài toán chia hoa hồng

- Hàng tháng mỗi người bán sẽ gửi báo cáo về cho cửa hàng với tổng số hàng bán được cho mỗi thị trấn anh ta đến
- Số thành phố đến được mỗi tháng là từ 1 đến 10.
- Người bán nhận được:
 - 10% nếu số tiền bán được ≤ 1000 đô,
 - 15% trên 800 đô tiếp theo, và
 - 20% của số tiền vượt quá 1800.



Các ca kiểm thử theo khoảng đầu ra





Các ca kiểm thử miền giá trị

Case #	Locks	Stocks	Barrels	Sales	Comm.	Comments
1	1	1	1	100	10	min
2	10	10	9	975	97.5	border-
3	10	9	10	970	97	border-
4	9	10	10	955	95.5	border-
5	10	10	10	1000	100	border
6	10	10	11	1025	103.75	border+
7	10	11	10	1030	104.5	border+
8	11	10	10	1045	106.75	border+

Các ca kiểm thử giá trị đặc biệt

TT	Khóa	Bảng	Nòng	Số bán	Hoa hồng	Ghi chú
1	10	11	9	1005	100.7 5	border +
2	18	17	19	1795	219.2 5	border +
3	18	19	17	1805	221	border +



Bài tập

- Áp dụng lý thuyết vào các chương trình cho các bài toán minh họa đã xây dựng



Nội dung

1. Kiểm thử hộp đen (kiểm thử chức năng)
 - Kiểm thử lớp tương đương
 - Kiểm thử giá trị biên
 - **Kiểm thử bảng quyết định**
 - Kiểm thử dựa trên đặc tả use cases
2. Kiểm thử hộp trắng (kiểm thử cấu trúc)
 - Kiểm thử luồng điều khiển
 - Kiểm thử luồng dữ liệu



Bảng quyết định

- Yêu cầu chức năng có thể mô tả bằng bảng quyết định (Decision Table - DT)
- DT là một cách chính xác và gọn để mô tả logic phức tạp
 - Gắn các điều kiện với các hành động tương ứng
 - Giống lệnh if-then-else và switch-case
- DT có thể liên kết nhiều điều kiện độc lập với vài hành động một cách dễ hiểu
 - Khác các cấu trúc điều khiển trong các ngôn ngữ lập trình



Cấu trúc bảng quyết định

	Rule 1	Rule 2	...	Rule p
Conditions				
Condition-1				
...				
Condition-m				

Actions				
Action-1				
...				
Action-n				



Cấu trúc bảng quyết định

1. Các điều kiện	2. Các giá trị điều kiện
3. Hành động	4. Xảy ra hay không

1. Mỗi điều kiện tương ứng với một biến, một quan hệ, hoặc một mệnh đề (predicate)
2. Các giá trị của điều kiện
 - Chỉ là True/False – Bảng quyết định hạn chế
 - Một số giá trị – Bảng quyết định mở rộng
 - Giá trị không quan tâm
3. Mỗi hành động là một thủ tục hoặc thao tác phải thực hiện
4. Đánh dấu hành động có/không xảy ra



Ví dụ về bảng quyết định

Điều kiện	Máy in không in	Y	Y	Y	Y	N	N	N	N
	Đèn đỏ nhấp nháy	Y	Y	N	N	Y	Y	N	N
	Không nhận ra máy in	Y	N	Y	N	Y	N	Y	N
Hành động	Kiểm tra dây nguồn			X					
	Kiểm tra dây tín hiệu	X		X					
	Kiểm tra phần mềm in đã cài đúng	X		X		X		X	
	Kiểm tra/thay mực	X	X			X	X		
	Kiểm tra kẹt giấy		X		X				

Khắc phục sự cố máy in



Ví dụ bảng quyết định tính chiết khấu

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Married?	Yes	Yes	No	No
Good Student?	Yes	No	Yes	No
Actions				
Discount (\$)	60	25	50	0



Sử dụng bảng quyết định

- Để quan sát tất cả các điều kiện dễ dàng
- Có thể dùng để
 - Mô tả logic phức tạp
 - Sinh ca kiểm thử, còn gọi là *kiểm thử dựa trên logic*
- Kiểm thử dựa trên logic được xem là:
 - Kiểm thử cấu trúc khi áp dụng cho các cấu trúc chương trình
 - Vd luồng điều khiển
 - Kiểm thử hàm khi áp dụng cho đặc tả.



Phương pháp xây dựng bảng

1. Xác định các điều kiện và giá trị của chúng
2. Xác định số luật tối đa
3. Xác định các hành động
4. Đánh số các luật nếu cần
5. Đánh số các hành động thích hợp cho mỗi luật
6. Kiểm tra chính sách
7. Đơn giản hóa các luật (gộp cột)



Thiết kế ca kiểm thử

- Khi đặc tả đã được kiểm tra, mục tiêu là chứng tỏ chương trình thực hiện các hành động đúng cho mọi tổ hợp các giá trị của mệnh đề
 - Nếu có k luật và n mệnh đề đúng/sai, thì có ít nhất k trường hợp và nhiều nhất là 2^n trường hợp phải xét.
- Có thể dựa trên các luật chưa mở rộng hoặc các luật đã mở rộng với 2^n ca
 - Xác định đầu vào cho mỗi ca



Thiết kế ca kiểm thử

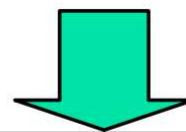
- Để xác định ca kiểm thử, chúng ta chuyển các điều kiện thành đầu vào, hành động thành đầu ra.
- Một số điều kiện sẽ tham chiếu đến các lớp tương đương đầu vào, và hành động tham chiếu đến các phần xử lý chức năng chính của cột đang xét.
- Các luật được chuyển thành các ca kiểm thử



Quy trình kiểm thử dựa trên bảng quyết định

1. Xây dựng bảng quyết định dựa trên đặc tả phần mềm
2. Từ bảng quyết định chuyển thành bảng các test cases trong đó mỗi cột miêu tả một luật được chuyển thành 1 đến n cột miêu tả các test cases tương ứng với luật đó
 1. Nếu điều kiện nhập là 1 giá trị thì mỗi cột luật được chuyển thành 1 cột test case
 2. Nếu điều kiện nhập là một lớp tương đương (nhiều giá trị liên tục) thì mỗi cột luật được chuyển thành nhiều test cases dựa trên kỹ thuật lớp tương đương hoặc kỹ thuật giá trị biên.

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Cond 1	Yes	Yes	No	No
Cond 2	Yes	No	Yes	No
Actions				
Action-1	60	25	50	0



	TC1	TC2	TC3	TC4
Conditions				
Married?	Yes	Yes	No	No
Good Student?	Yes	No	Yes	No
Actions				
Discount (\$)	60	25	50	0

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Cond 1	0–1	1–10	10–100	100–1000
Cond 2	<5	5	6 or 7	>7
Actions				
Action-1	Do X	Do Y	Do X	Do Z



(Áp dụng kỹ thuật lớp tương đương để chọn giá trị đầu vào?)

	TC1	TC2	TC3	TC4
Conditions				
Cond 1	0	5	50	500
Cond 2	3	5	7	10
Actions				
Action-1	Do X	Do Y	Do X	Do Z



Sử dụng bảng quyết định

- Bảng thích hợp khi:
 - Đặc tả có thể chuyển về dạng bảng
 - Thứ tự các hành động xảy ra không quan trọng
 - Thứ tự các luật không ảnh hưởng đến hành động
 - Khi một luật thỏa mãn và được chọn thì không cần xét luật khác
- Các hạn chế trên không ảnh hưởng đến việc sử dụng bảng
 - Trong hầu hết các ứng dụng thứ tự các mệnh đề được xét là không quan trọng



Một số vấn đề với bảng quyết định

- Trước khi sử dụng bảng cần đảm bảo:
 - Các luật phải đầy đủ
 - Có mọi tổ hợp
 - Các luật phải nhất quán
 - Mọi tổ hợp giá trị chân lý chỉ gây ra một hoặc một tập hành động



Bảng quyết định cho Triangle

Conditions											
C1: $a < b+c?$	F	T	T	T	T	T	T	T	T	T	T
C2: $b < a+c?$	-	F	T	T	T	T	T	T	T	T	T
C3: $c < a+b?$	-	-	F	T	T	T	T	T	T	T	T
C4: $a=b?$	-	-	-	T	T	T	T	F	F	F	F
C5: $a=c?$	-	-	-	T	T	F	F	T	T	F	F
C6: $b=c?$	-	-	-	T	F	T	F	T	F	T	F
Actions											
A1: Not a Triangle	X	X	X								
A2: Scalene											X
A3: Isosceles							X		X	X	
A4: Equilateral				X							
A5: Impossible					X	X		X			



Ca kiểm thử cho Triangle

Case ID	a	b	c	Expected Output
DT1	4	1	2	Not a Triangle
DT2	1	4	2	Not a Triangle
DT3	1	2	4	Not a Triangle
DT4	5	5	5	Equilateral
DT5	?	?	?	Impossible
DT6	?	?	?	Impossible
DT7	2	2	3	Isosceles
DT8	?	?	?	Impossible
DT9	2	3	2	Isosceles
DT10	3	2	2	Isosceles
DT11	3	4	5	Scalene / Cân



Sử dụng DT để xây dựng Testcase

- Áp dụng với bài toán Tuyển dụng nhân sự
- Tính lương theo mức lương tối thiểu vùng

=> tạo các TC



Bảng quyết định cho NextDate (thử lần 1)

- Chúng ta có thể chia thành các lớp tương đương sau:

M1= {month | month has 30 days}

M2= {month | month has 31 days}

M3= {month | month is February}

D1= {day | $1 \leq \text{day} \leq 28$ }

D2= {day | day = 29}

D3= {day | day = 30}

D4= {day | day=31}

Y1= {year | year = 1900}

Y2= {year | $1812 \leq \text{year} \leq 2012$ AND $\text{year} \neq 1900$ AND $(0 = \text{year mod } 4)$ }

Y3= {year | $1812 \leq \text{year} \leq 2012$ AND $0 \neq \text{year mod } 4$ }

- Khi đó ta có bảng quyết định như sau



Bảng quyết định cho NextDate (1)

Conditions	1	2	3	4	5	6	7	8
C1: month in	M1	M1	M1	M1	M2	M2	M2	M2
C2: day in	D1	D2	D3	D4	D1	D2	D3	D4
C3: year in	-	-	-	-	-	-	-	-
Rule count	3	3	3	3	3	3	3	3
Actions								
A1: Impossible				X				
A2: Increment day	X	X			X	X	X	
A3: Reset day			X					X
A4: Increment month			X					?
A5: reset month								?
A6: Increment year								?



Bảng quyết định cho NextDate (2)

Conditions	9	10	11	12	13	14	15	16
C1: month in	M3	M3	M3	M3	M3	M3	M3	M3
C2: day in	D1	D1	D1	D2	D2	D2	D3	D3
C3: year in	Y1	Y2	Y3	Y1	Y2	Y3	-	-
Rule count	1	1	1	1	1	1	3	3
Actions								
A1: Impossible				X		X	X	X
A2: Increment day		X						
A3: Reset day	X		X		X			
A4: Increment month	X		X		X			
A5: reset month								
A6: Increment year								



Bảng quyết định cho NextDate (Lần thứ 2)

■ Xét một cách phân hoạch khác:

M1= {month | month has 30 days}
M2= {month | month has 31 days}
M3= {month | month is December}
M4= {month | month is February}

D1= {day | $1 \leq \text{day} \leq 27$ }
D2= {day | day = 28}
D3= {day | day = 29}
D4= {day | day = 30}
D5= {day | day=31}

Y1= {year | year is a leap year}
Y2= {year | year is a common year}

M1= {month | month has 30 days}
M2= {month | month has 31 days}
M3= {month | month is February}

D1= {day | $1 \leq \text{day} \leq 28$ }
D2= {day | day = 29}
D3= {day | day = 30}
D4= {day | day=31}

Y1= {year | year = 1900}
Y2= {year | $1812 \leq \text{year} \leq 2012$
AND year $\neq$ 1900
AND $(0 = \text{year mod } 4)$ }
Y3= {year | $1812 \leq \text{year} \leq 2012$
AND $0 \neq \text{year mod } 4$ }

[illegible]



Bảng quyết định cho NextDate (2)

Conditions	11	12	13	14	15	16	17	18	19	20	21	22
C1: month in	M3	M3	M3	M3	M3	M4	M4	M4	M4	M4	M4	M4
C2: day in	D1	D2	D3	D4	D5	D1	D2	D2	D3	D3	D4	D5
C3: year in	-	-	-	-	-	-	Y1	Y2	Y1	Y2	-	-
Actions												
A1: Impossible										X	X	X
A2: Increment day	X	X	X	X		X	X					
A3: Reset day					X			X	X			
A4: Increment month								X	X			
A5: reset month					X							
A6: Increment year					X							



Quan sát và hướng dẫn

- Bảng quyết định phù hợp khi
 - Có nhiều quyết định đưa ra
 - Có các quan hệ logic quan trọng giữa các biến đầu vào
 - Có các tính toán liên quan đến các tập con của các biến đầu vào
 - Có quan hệ nhân quả giữa đầu vào và đầu ra
 - Có logic tính toán phức tạp (độ phức tạp đồ thị cyclomatic cao)
- Bảng quyết định không dễ mở rộng (scale up)
- Bảng quyết định có thể làm mịn, cải tiến dần



Bài tập

- Ứng dụng kiểm thử bảng quyết định cho một số bài toán ví dụ



CS8

- Dựa trên bài thảo luận
 - Xác định 1 chức năng
 - Xác định các lớp tương đương (liên quan đến giá trị đầu vào)
 - Xác định số lượng ca kiểm thử tương ứng
 - Xây dựng một số ca kiểm thử dựa trên:
 - Lớp tương đương
 - Giá trị biên

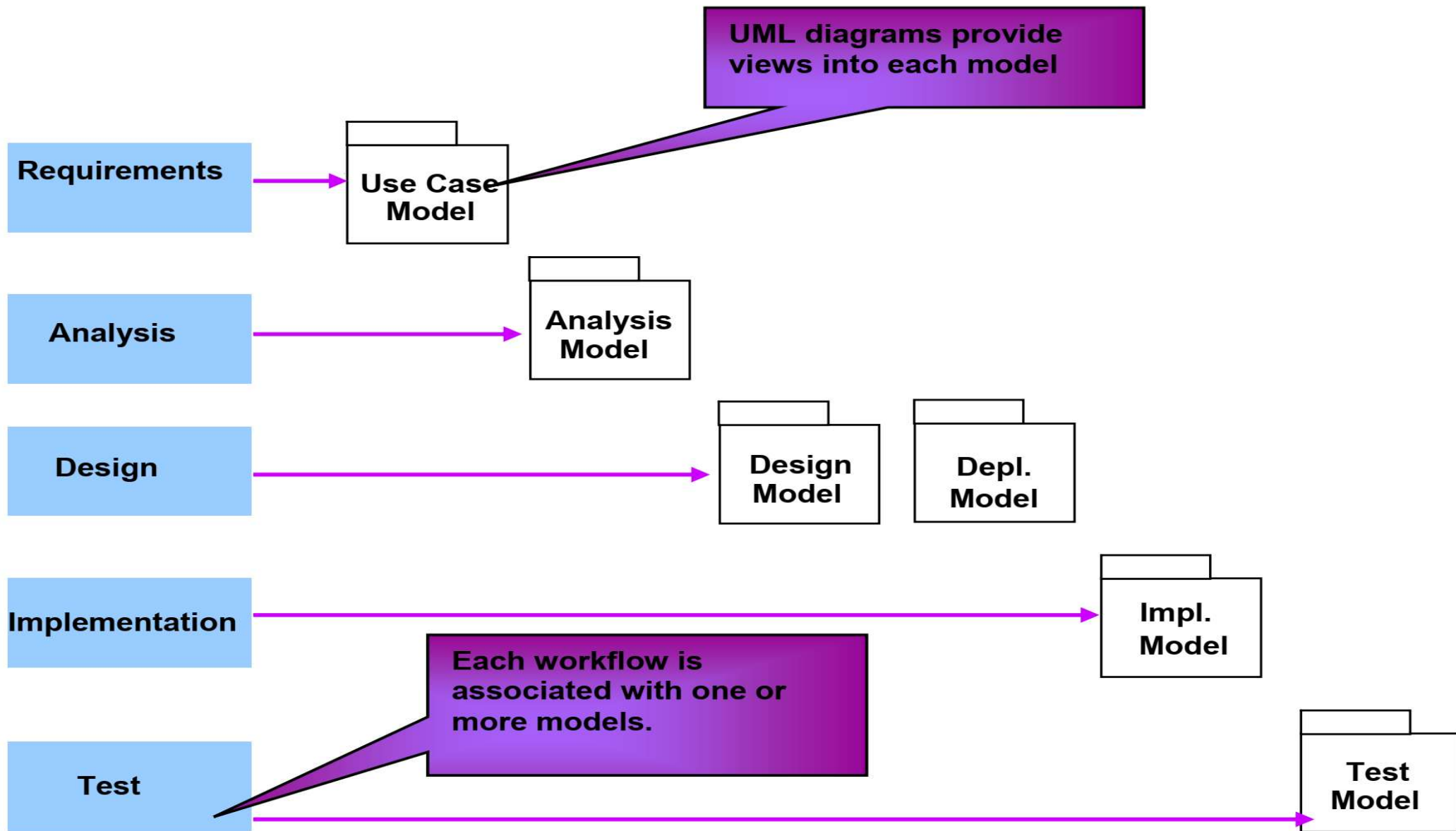


Nội dung

1. Kiểm thử hộp đen (kiểm thử chức năng)
 - Kiểm thử lớp tương đương
 - Kiểm thử giá trị biên
 - Kiểm thử bảng quyết định
 - **Kiểm thử dựa trên đặc tả use cases**
2. Kiểm thử hộp trắng (kiểm thử cấu trúc)
 - Kiểm thử luồng điều khiển
 - Kiểm thử luồng dữ liệu



Quy trình phát triển phần mềm (hướng đối tượng)





Kiểm thử dựa trên đặc tả use cases

- Mỗi use case thể hiện một chức năng mà người dùng thực hiện
- Mô hình use cases gồm:
 - Biểu đồ use cases
 - Đặc tả chi tiết use cases (use case specifications)
- Mô tả phần mềm theo góc nhìn từ bên ngoài (xem phần mềm như hộp đen)



Đặc tả test cases

- Luồng chính (main flow/basic flow)
- Luồng phụ/luồng ngoại lệ/luồng mở rộng (alternation flows/extensions)

Use Case Component	Description	
Use Case Number or Identifier	SURS1138	
Use Case Name	Register for a course (a class taught by a faculty member)	
Goal in Context		
Scope	System	
Level	Primary task	
Primary Actor	Student	
Preconditions	None	
Success End Conditions	The student is registered for the course—the course has been added to the student's course list	
Failed End Conditions	The student's course list is unchanged	
Trigger	Student selects a course and "Registers"	
Main Success Scenario A: Actor S: System	Step	Action
	1	A: Selects "Register for a course"
	2	A: Selects course (e.g. Math 1060)
	3	S: Displays course description
	4	A: Selects section (Mon & Wed 9:00am)
	5	S: Displays section days and times
	6	A: Accepts
Extensions	7	S: Adds course/section to student's course list
	2a	Course does not exist S: Display message and exit
	4a	Section does not exist S: Display message and exit
	4b	Section is full S: Display message and exit
	6a	Student does not accept S: Display message and exit



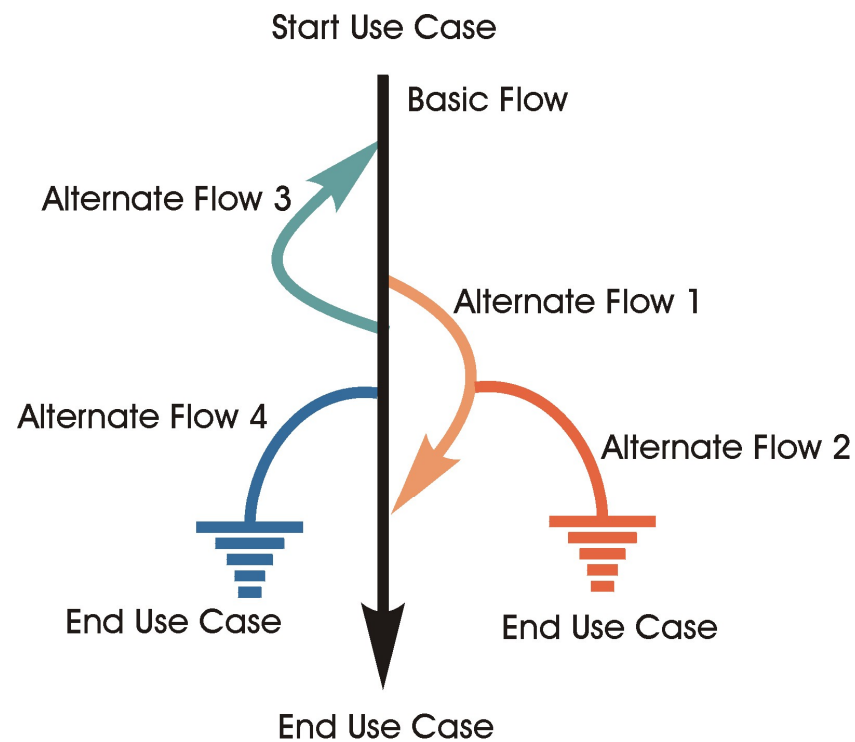
Quy trình thực hiện

1. Xác định các luồng sự kiện vào ra giữa tác nhân và hệ thống trong đặc tả use cases
2. Xác định các kịch bản hành vi
3. Xác định các test cases



Deriving Test Cases from Use Cases

- Xác định các luồng, các bước trong mỗi luồng





Deriving Test Cases from Use Cases

- **Xác định các kịch bản hành vi (behavior scenarios)**

Scenario 1	Basic Flow			
Scenario 2	Basic Flow	Alternate Flow 1		
Scenario 3	Basic Flow	Alternate Flow 1	Alternate Flow 2	
Scenario 4	Basic Flow	Alternate Flow 3		
Scenario 5	Basic Flow	Alternate Flow 3	Alternate Flow 1	
Scenario 6	Basic Flow	Alternate Flow 3	Alternate Flow 1	Alternate Flow 2
Scenario 7	Basic Flow	Alternate Flow 4		
Scenario 8	Basic Flow	Alternate Flow 3	Alternate Flow 4	



Deriving Test Cases from Use Cases

- **Xác định các test cases cho mỗi kịch bản hành vi bằng cách xác định các điều kiện cụ thể (condition) kích hoạt hệ thống thực hiện hành vi đó**

Test Case ID	Scenario	Condition	Expected Result
TC x	Scenario 4	Step 2 - Withdraw Amount > Account Balance	Rejoin basic flow at Step 2
TC y	Scenario 4	Step 2 - Withdraw Amount < Account Balance	Does not execute Alternate Flow 3, takes basic flow
TC z	Scenario 4	Step 2 - Withdraw Amount = Account Balance	Does not execute Alternate Flow 3, takes basic flow

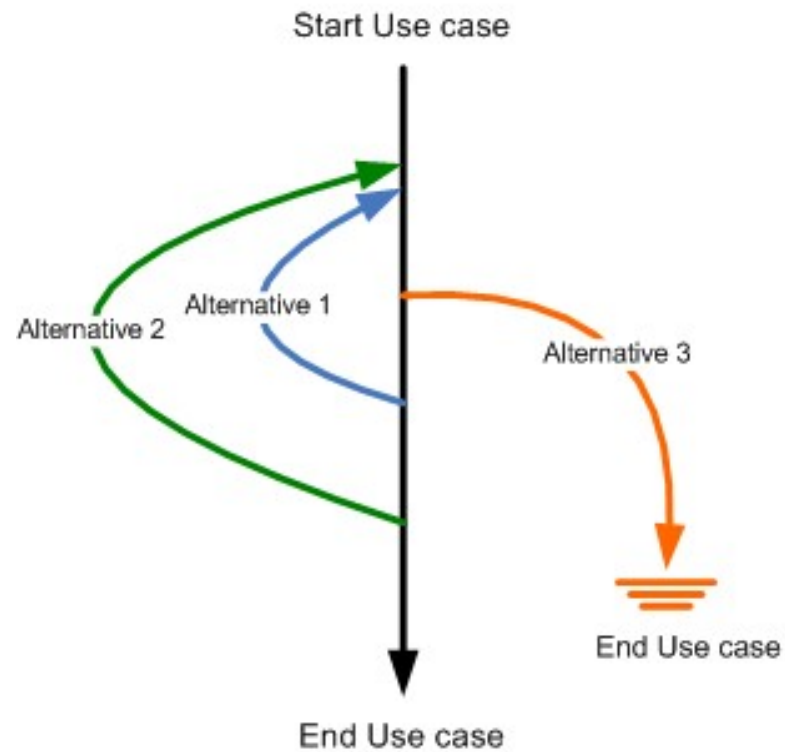


Example - Deriving Test Cases from Use Cases

Basic Flow	This Use Case allows creating a Department with Department Name field. Department Name length is from 3 to 20 characters and only includes letter characters and number characters. Steps: 1. Initiates a blank form to input Department Name field 2. Verifies validity of input characters 3. Verifies Department Name coincidence 4. A Department is added to system and System displays message "Creating Department is successful!" Use Case ends with a blank form for input another Department.
-------------------	--

Example - Deriving Test Cases from Use Cases

- Identify Flows





Example - Deriving Test Cases from Use Cases

□ Identify Flows

Alternate Flow 1 - Not a valid Name	In Basic Flow Step 2: Verify Name, if Data is not valid, System displays an error message.
Alternate Flow 2 – Department is coincident	At Basic Flow Step 3: Verify Department coincidence, if Department is exist, System displays an error message.
Alternate Flow 3 - Quit	At any time, user can decide to quit.



Example - Deriving Test Cases from Use Cases

- Identify Use case scenarios:

Scenario 1	Basic Flow		
Scenario 2	Basic Flow	Alternate Flow 1	
Scenario 3	Basic Flow	Alternate Flow 1	Alternate Flow 3
Scenario 4	Basic Flow	Alternate Flow 2	
Scenario 5	Basic Flow	Alternate Flow 2	Alternate Flow 1
Scenario 6	Basic Flow	Alternate Flow 2	Alternate Flow 3



Example - Deriving Test Cases from Use Cases

- Deriving the test cases for each scenario
- Example with scenario 2:

Test Case ID	Scenario	Condition	Expected Result
TC x	Scenario 2	Step 2 – Name length > 20 characters	Rejoin basic flow at Step 2
TC y	Scenario 2	Step 2 - Name length < 3 characters	Rejoin basic flow at Step 2
TC z	Scenario 2	Step 2 – Name includes invalid characters	Rejoin basic flow at Step 2



Deriving Test Cases from Use Cases

When identifying functional test cases, ensure the following:

- sufficient test cases, positive and negative for each use-case scenario
- test cases address any business rules
- test cases address any sequencing of events or actions.
- test cases address any special requirements defined for the use case (performance, loads or data volumes)



Vận dụng:

UC: CREATE ACCOUNT

Brief Description

This use case describes how to create account in this system. Actor of this use case is the Guest

Pre-conditions

There are no preconditions associated with this use case

Flow of events

Basic Flow

1. Initiates a blank form to creating account. The user enters information into fields: Name, Password, Date of Birth. Then user selects "Submit" button
2. The system validates the data insure the proper data format
3. A new account is added to system. The system displays message:" Your Account Has Been Created". The user selects "OK" button. And the use case ends.

Special requirements

1. The required data fields display "*" character behind and must enter data. Name, Password are the required fields
2. The Name must contain from 3 to 10 characters. And name includes only letter characters and number characters
3. The Date of Birth must be in this format: mm/dd/yyyy
4. The Password must contain from 6 to 20 characters. And name includes only letter characters and number characters
5. Names of users must be different (to avoid account coincidence)



Identify flows:

Alternate Flow 1 - Not a valid Name, Password, and Date of birth	In Basic Flow Step 2: Verify Name, Password, and Date of birth, if Data is not valid, System displays an error message.
Alternate Flow 2 – Account is coincident	At Basic Flow Step 3: Verify Name coincidence, if account is <u>exist</u> , System displays an error message.
Alternate Flow 3 - Quit	At any time, user can decide to quit.

Identify scenarios:

Identify TCs for each scenario:

Test Case ID	Scenario	Condition	Expected Result



Bài tập:

- Mô tả use case thanh toán tại bàn thu ngân ở siêu thị
- Thực hiện các bước đã học để xác định các test cases cho use case trên.



CS9

- Chọn 1 chức năng trong sản phẩm của bài thảo luận, thực hiện:
 - ✓ Tạo use case
 - ✓ Xây dựng các kịch bản hành vi
 - ✓ Xây dựng các test case

